
Markov Decision Process

– |

CS5491: Artificial Intelligence

ZHICHAO LU

Content Credits: [Prof. Wei's CS4486 Course](#)
and [Prof. Boddeti's AI Course](#)



TODAY

Markov Decision Processes (MDPs)



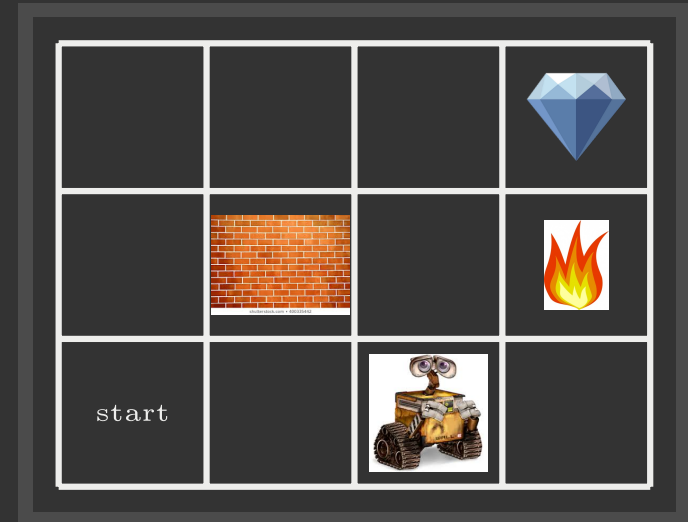
NON-DETERMINISTIC SEARCH



EXAMPLE: GRID WORLD

A maze like problem

- The agent lives in a grid
- Walls block the agent's path



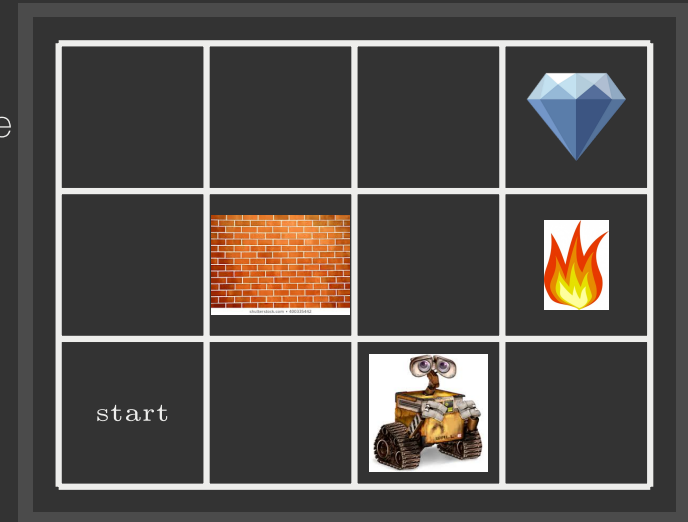
EXAMPLE: GRID WORLD

A maze like problem

- The agent lives in a grid
- Walls block the agent's path

Noisy movement: actions do not always go as planned

- 80% of the time, the action North takes the agent North (if there is no wall there)
- 10% of the time, North takes the agent West; 10% East
- If there is a wall in the direction the agent would have been taken, the agent stays put



EXAMPLE: GRID WORLD

A maze like problem

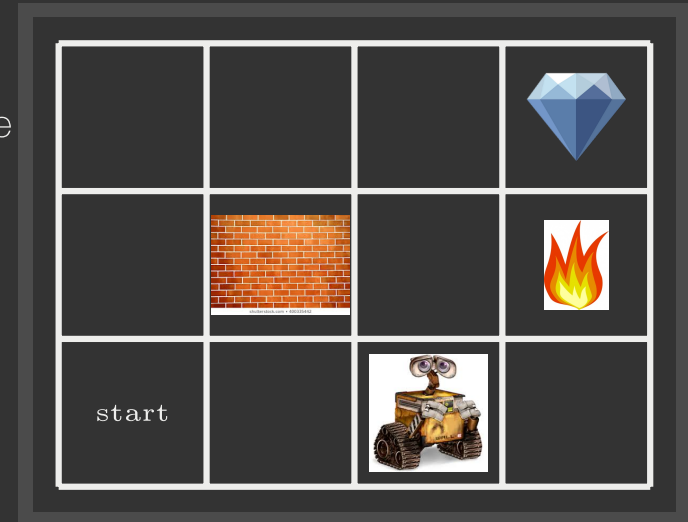
- The agent lives in a grid
- Walls block the agent's path

Noisy movement: actions do not always go as planned

- 80% of the time, the action North takes the agent North (if there is no wall there)
- 10% of the time, North takes the agent West; 10% East
- If there is a wall in the direction the agent would have been taken, the agent stays put

The agent receives rewards each time step

- Small "living" reward each step (can be negative)
- Big rewards come at the end (good or bad)



EXAMPLE: GRID WORLD

A maze like problem

- The agent lives in a grid
- Walls block the agent's path

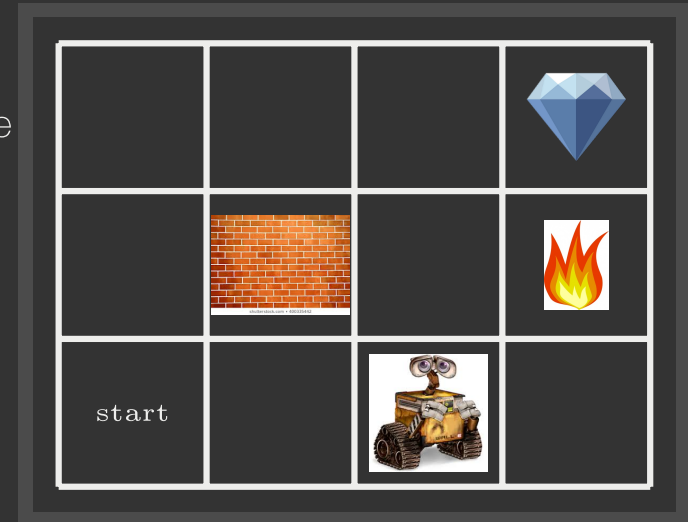
Noisy movement: actions do not always go as planned

- 80% of the time, the action North takes the agent North (if there is no wall there)
- 10% of the time, North takes the agent West; 10% East
- If there is a wall in the direction the agent would have been taken, the agent stays put

The agent receives rewards each time step

- Small "living" reward each step (can be negative)
- Big rewards come at the end (good or bad)

Goal: maximize sum of rewards



GRID WORLD ACTIONS

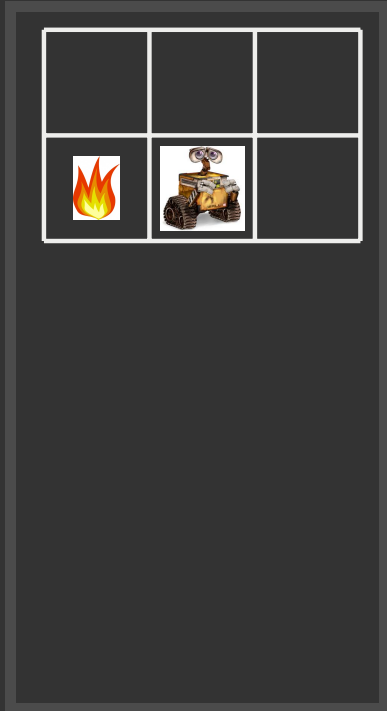
Deterministic Grid World

Stochastic Grid World



GRID WORLD ACTIONS

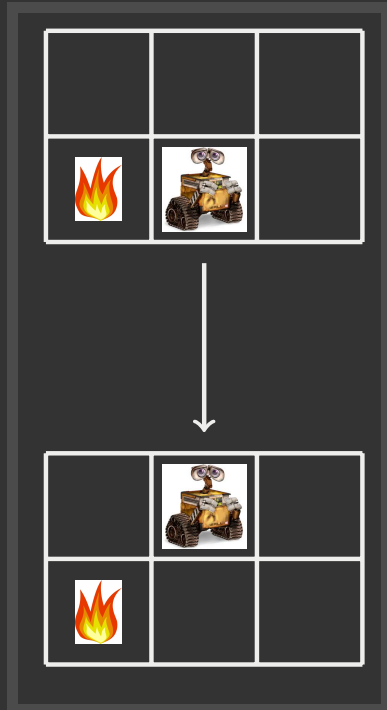
Deterministic Grid World



Stochastic Grid World

GRID WORLD ACTIONS

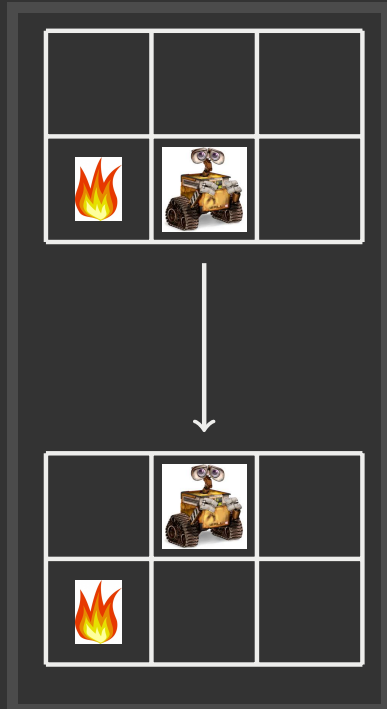
Deterministic Grid World



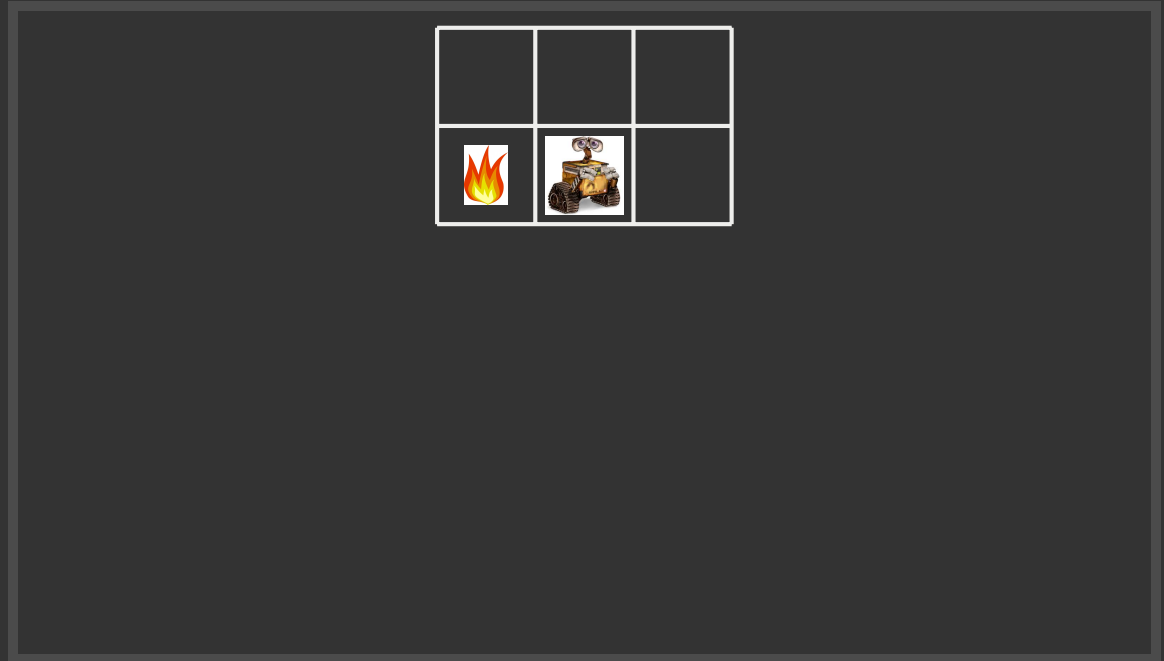
Stochastic Grid World

GRID WORLD ACTIONS

Deterministic Grid World

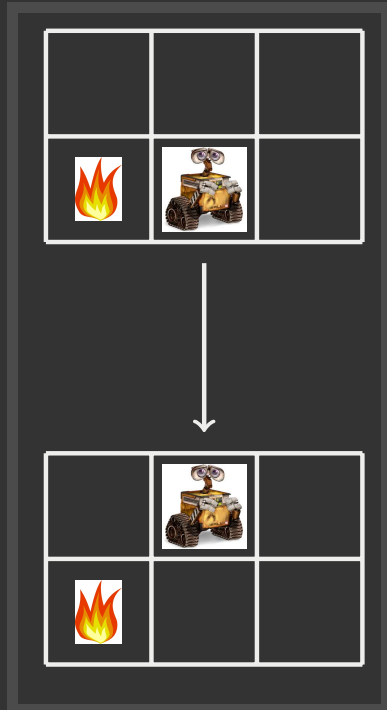


Stochastic Grid World

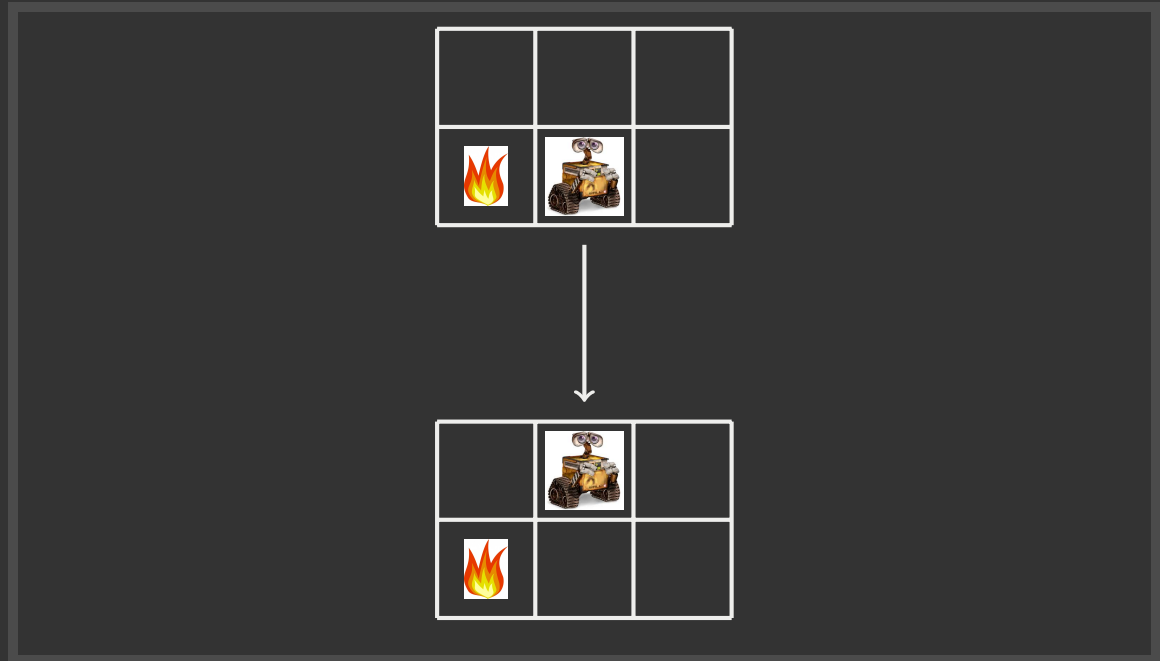


GRID WORLD ACTIONS

Deterministic Grid World

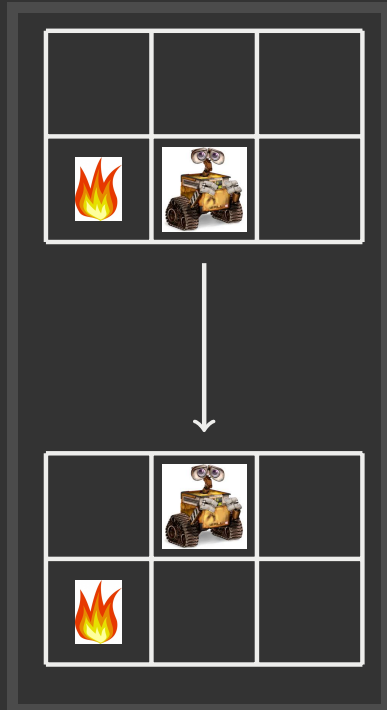


Stochastic Grid World

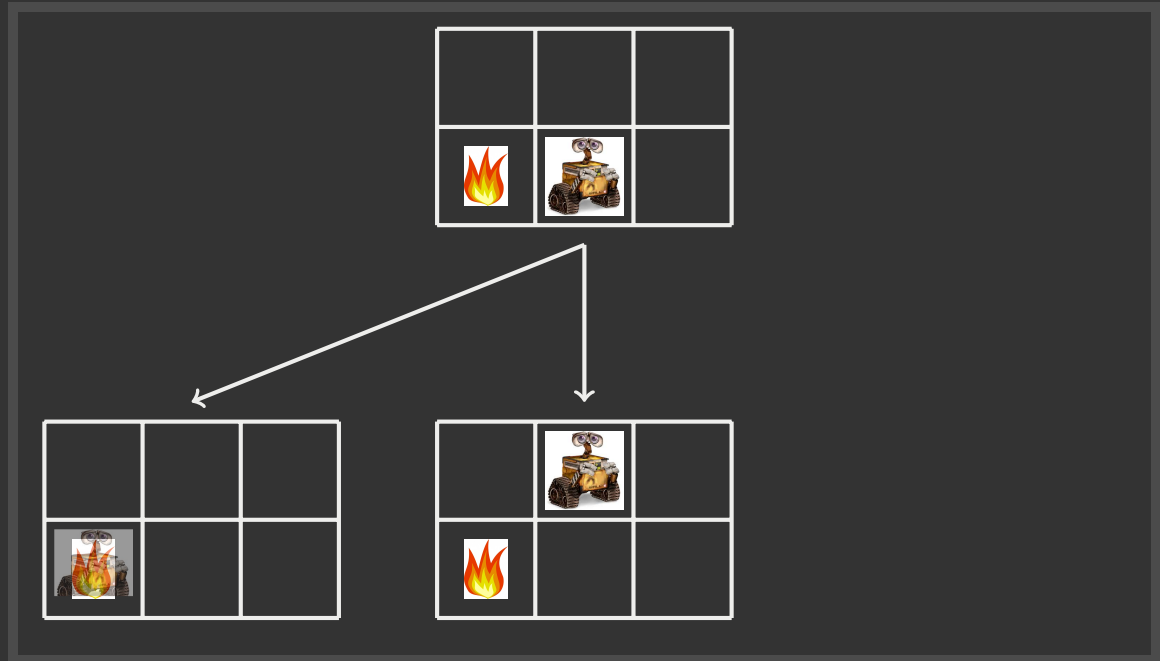


GRID WORLD ACTIONS

Deterministic Grid World

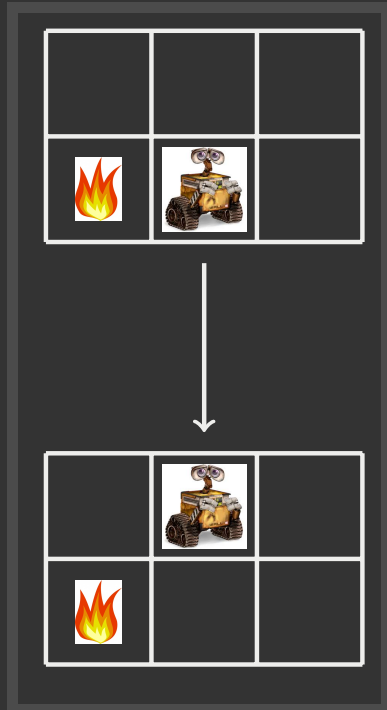


Stochastic Grid World

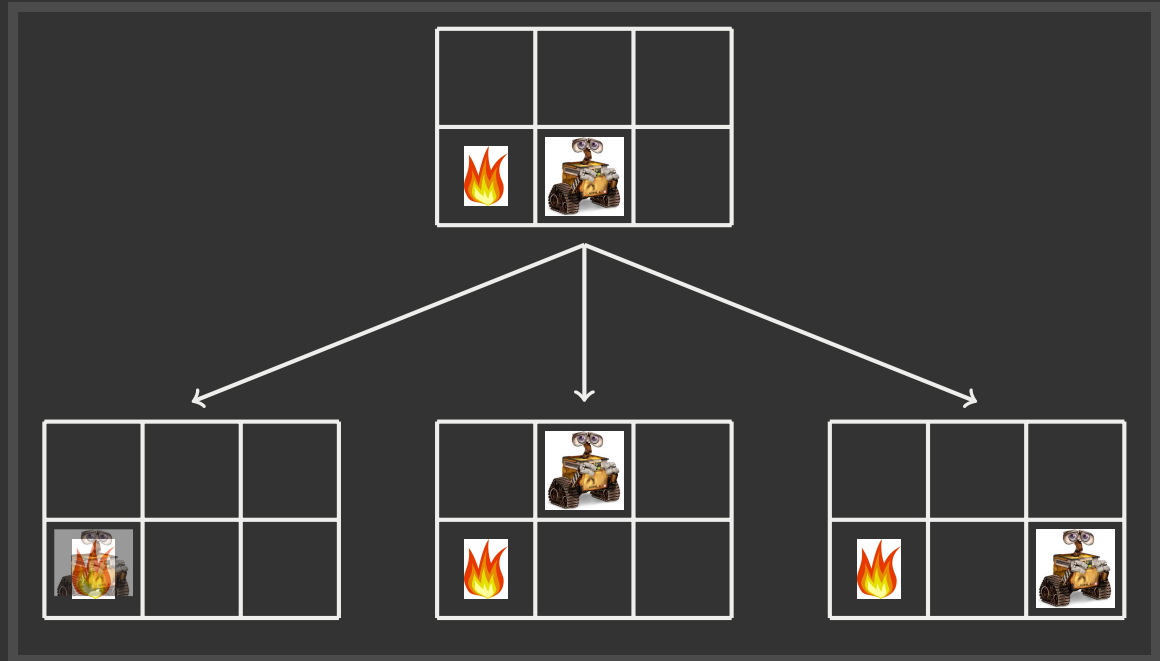


GRID WORLD ACTIONS

Deterministic Grid World

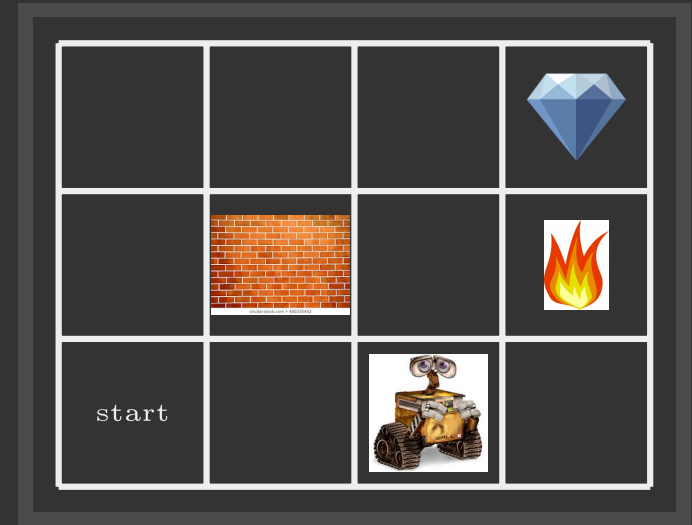


Stochastic Grid World



MARKOV DECISION PROCESSES

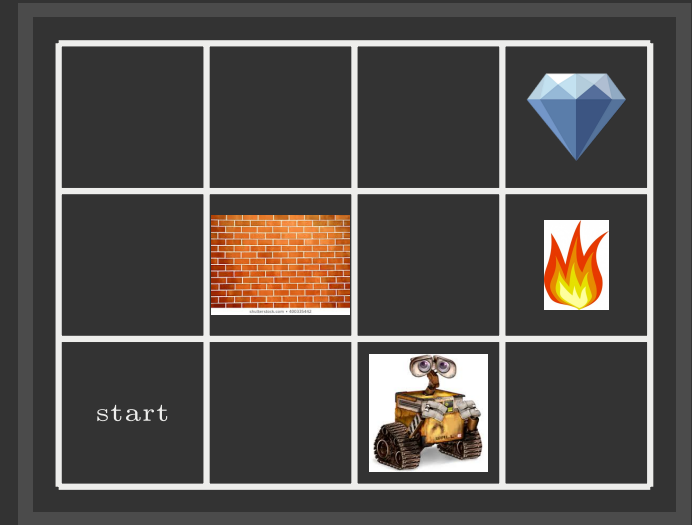
An MDP is defined by:



MARKOV DECISION PROCESSES

An MDP is defined by:

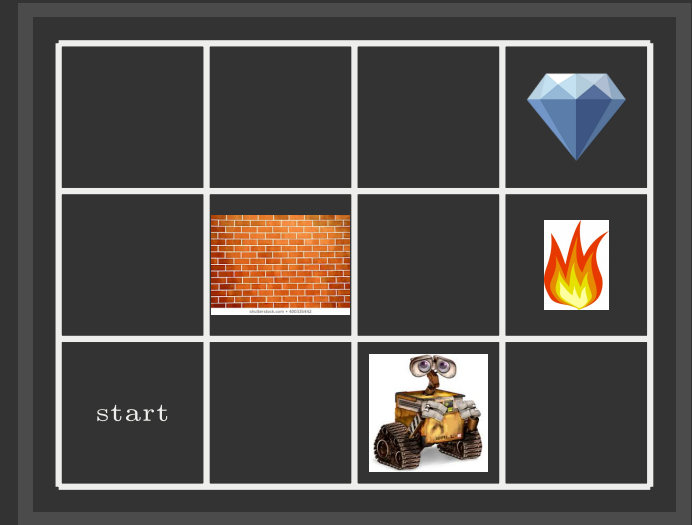
- A set of states $s \in \mathcal{S}$



MARKOV DECISION PROCESSES

An MDP is defined by:

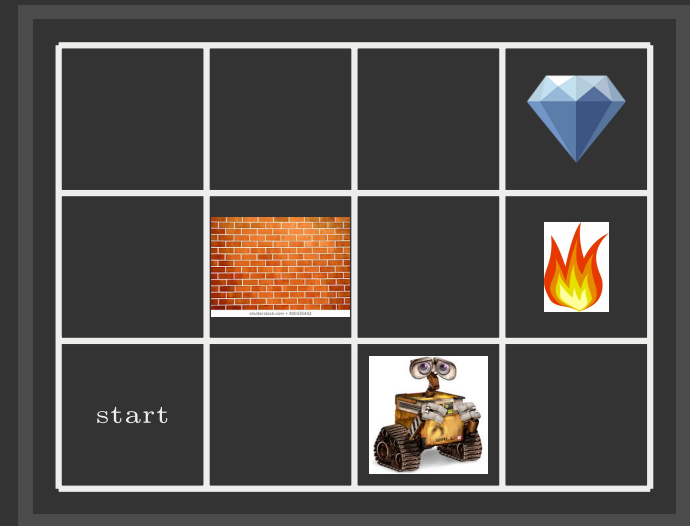
- A set of states $s \in \mathcal{S}$
- A set of actions $a \in \mathcal{A}$



MARKOV DECISION PROCESSES

An MDP is defined by:

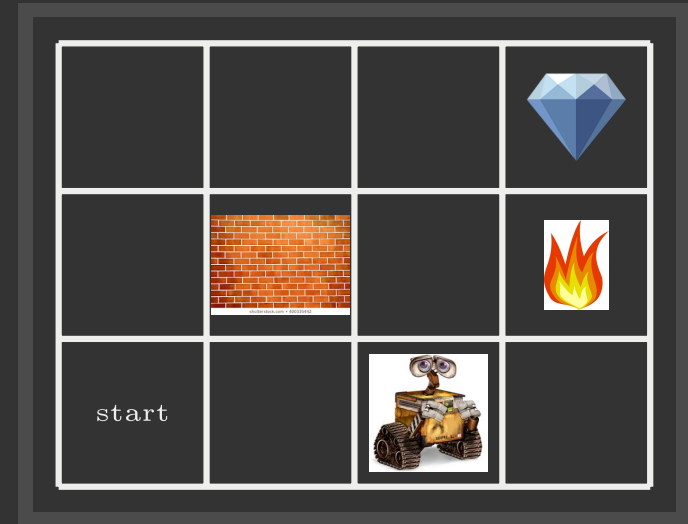
- A set of states $s \in \mathcal{S}$
- A set of actions $a \in \mathcal{A}$
- A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics



MARKOV DECISION PROCESSES

An MDP is defined by:

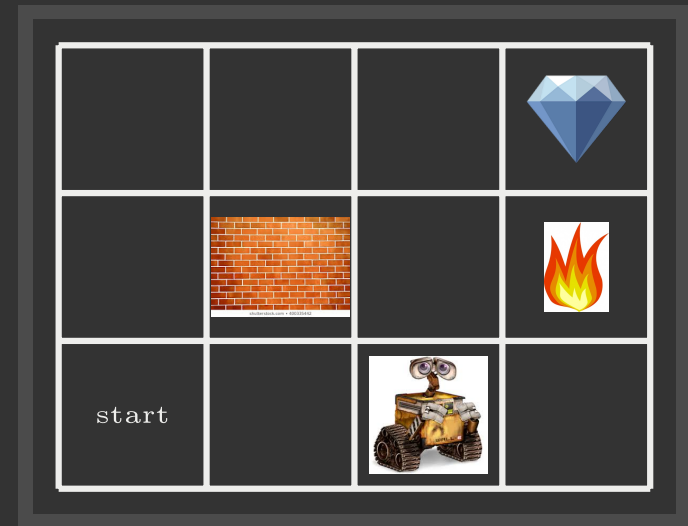
- A set of states $s \in \mathcal{S}$
- A set of actions $a \in \mathcal{A}$
- A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
- A reward function $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$



MARKOV DECISION PROCESSES

An MDP is defined by:

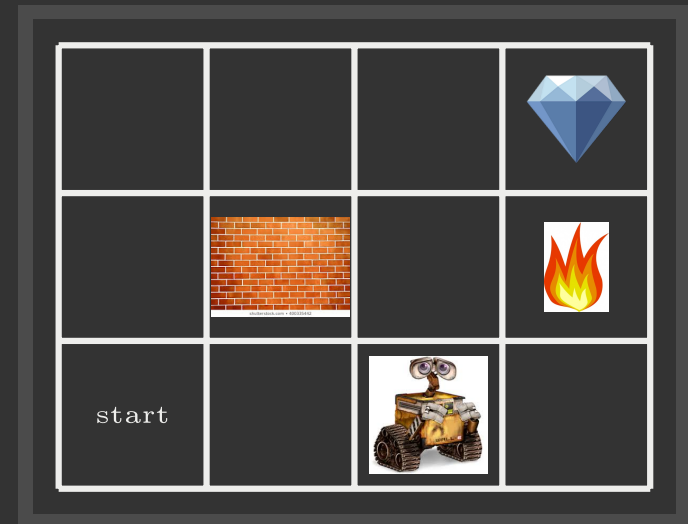
- A set of states $s \in \mathcal{S}$
- A set of actions $a \in \mathcal{A}$
- A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
- A reward function $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$
- A start state, s_0



MARKOV DECISION PROCESSES

An MDP is defined by:

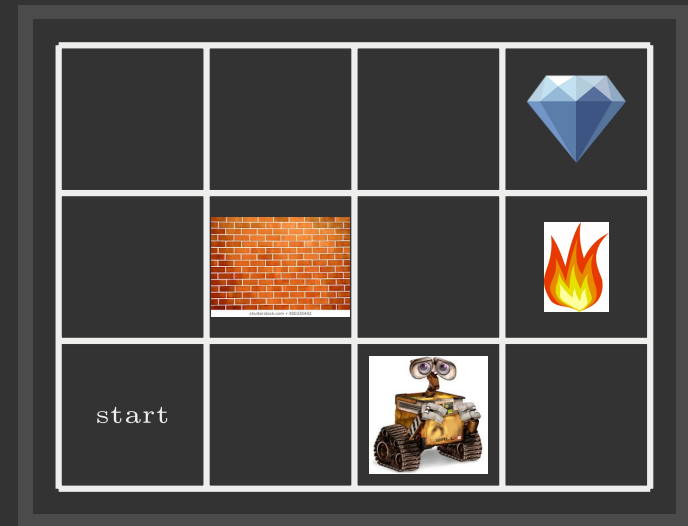
- A set of states $s \in \mathcal{S}$
- A set of actions $a \in \mathcal{A}$
- A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
- A reward function $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$
- A start state, s_0
- Maybe a terminal state



MARKOV DECISION PROCESSES

An MDP is defined by:

- A set of states $\mathbf{s} \in \mathcal{S}$
- A set of actions $\mathbf{a} \in \mathcal{A}$
- A transition function $T(\mathbf{s}, \mathbf{a}, \mathbf{s}')$
 - Probability that $\mathbf{a}$ from $\mathbf{s}$ leads to $\mathbf{s}'$, i.e., $P(\mathbf{s}'|\mathbf{s}, \mathbf{a})$
 - Also called the model or the dynamics
- A reward function $R(\mathbf{s}, \mathbf{a}, \mathbf{s}')$
 - Sometimes just $R(\mathbf{s})$ or $R(\mathbf{s}')$
- A start state, $\mathbf{s}_0$
- Maybe a terminal state



MDPs are non-deterministic search problems

- One way to solve them is with expectimax search
- We will have a new tool soon



WHAT IS MARKOV ABOUT MDPS

"Markov" generally means that given the present state, the future and the past are independent



Andrey Markov (1856–
1922)

WHAT IS MARKOV ABOUT MDPS

"Markov" generally means that given the present state, the future and the past are independent

For Markov decision processes, "Markov" means action outcomes depend only on the current state

$$\begin{aligned} p(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0) \\ = p(S_{t+1} = s' | S_t = s_t, A_t = a_t) \end{aligned}$$



Andrey Markov (1856–
1922)

WHAT IS MARKOV ABOUT MDPS

"Markov" generally means that given the present state, the future and the past are independent

For Markov decision processes, "Markov" means action outcomes depend only on the current state

$$\begin{aligned} p(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0) \\ = p(S_{t+1} = s' | S_t = s_t, A_t = a_t) \end{aligned}$$

This is just like search, where the successor function could only depend on the current state (not the history)



Andrey Markov (1856–
1922)

POLICIES

In deterministic single-agent search problems, we wanted an optimal plan, or sequence of actions, from start to a goal



Optimal policy when
 $R(s, a, s') = -0.01$ for all
non-terminals s .

POLICIES

In deterministic single-agent search problems, we wanted an optimal plan, or sequence of actions, from start to a goal

For MDPs, we want an optimal policy $\pi^* : \mathcal{S} \rightarrow \mathcal{A}$

- A policy π gives an action for each state
- An optimal policy is one that maximizes expected utility if followed
- An explicit policy defines a reflex agent



Optimal policy when
 $R(s, a, s') = -0.01$ for all
non-terminals s .

POLICIES

In deterministic single-agent search problems, we wanted an optimal plan, or sequence of actions, from start to a goal

For MDPs, we want an optimal policy $\pi^* : \mathcal{S} \rightarrow \mathcal{A}$

- A policy π gives an action for each state
- An optimal policy is one that maximizes expected utility if followed
- An explicit policy defines a reflex agent

Expectimax did not compute entire policies

- It computed the action for a single state only



Optimal policy when
 $R(s, a, s') = -0.01$ for all
non-terminals s .

OPTIMAL POLICIES

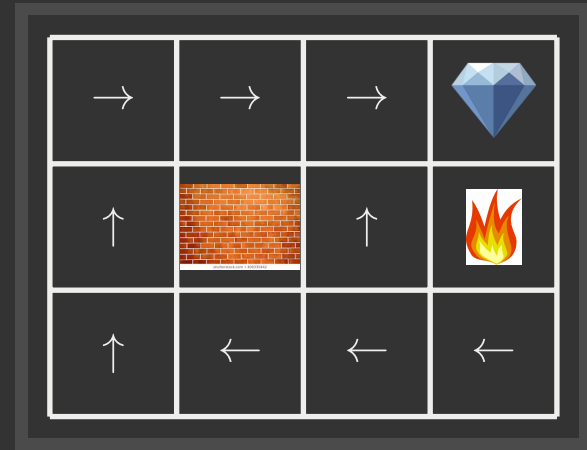


$$R(s) = -0.01$$

OPTIMAL POLICIES



$$R(s) = -0.01$$



$$R(s) = -0.03$$

OPTIMAL POLICIES



$$R(s) = -0.01$$



$$R(s) = -0.03$$



$$R(s) = -0.4$$



OPTIMAL POLICIES



$$R(s) = -0.01$$



$$R(s) = -0.03$$



$$R(s) = -0.4$$

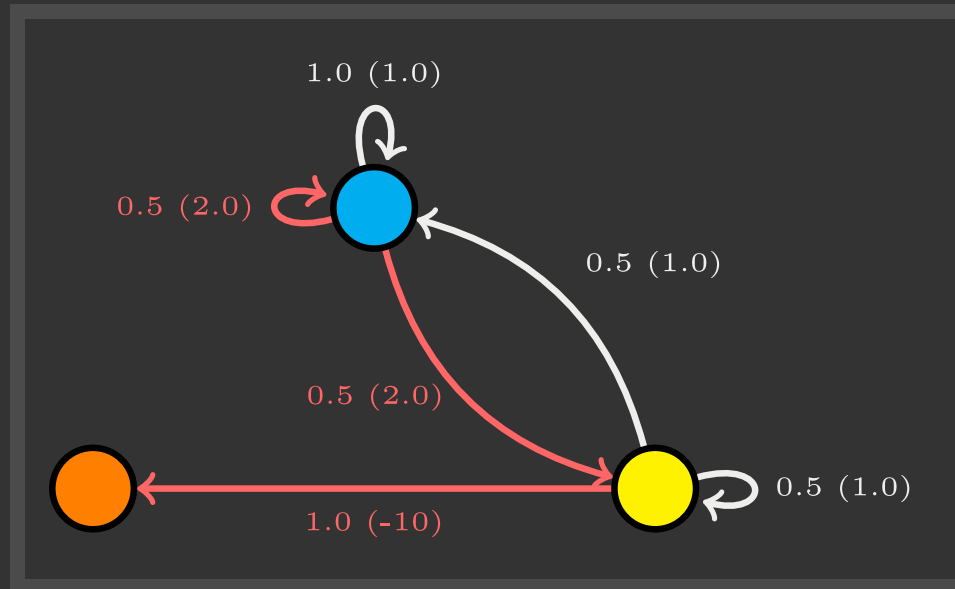


$$R(s) = -2.0$$



EXAMPLE: CAR RACING

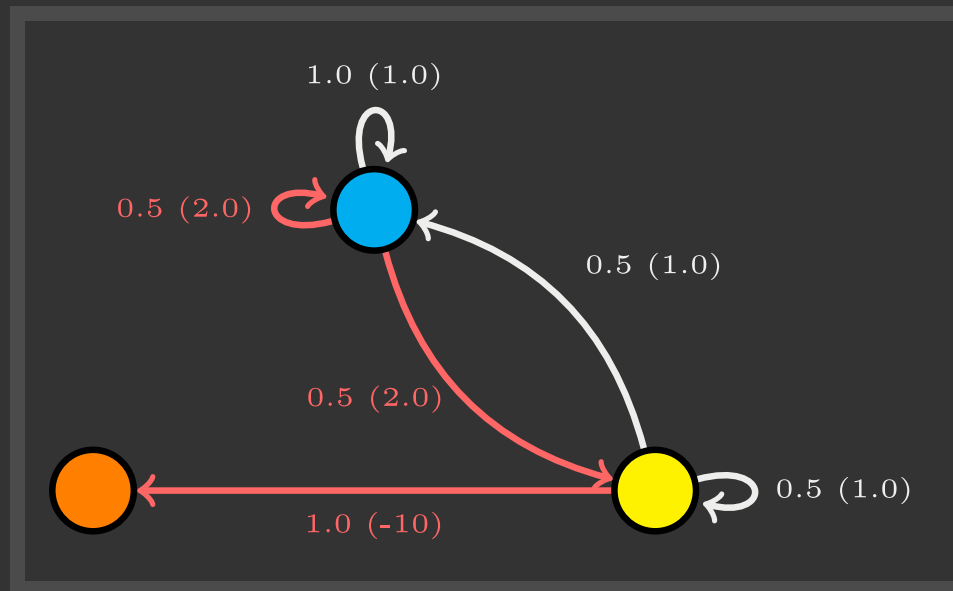
A robot car wants to travel far, quickly



EXAMPLE: CAR RACING

A robot car wants to travel far, quickly

Three states: Cool, Warm, Overheated

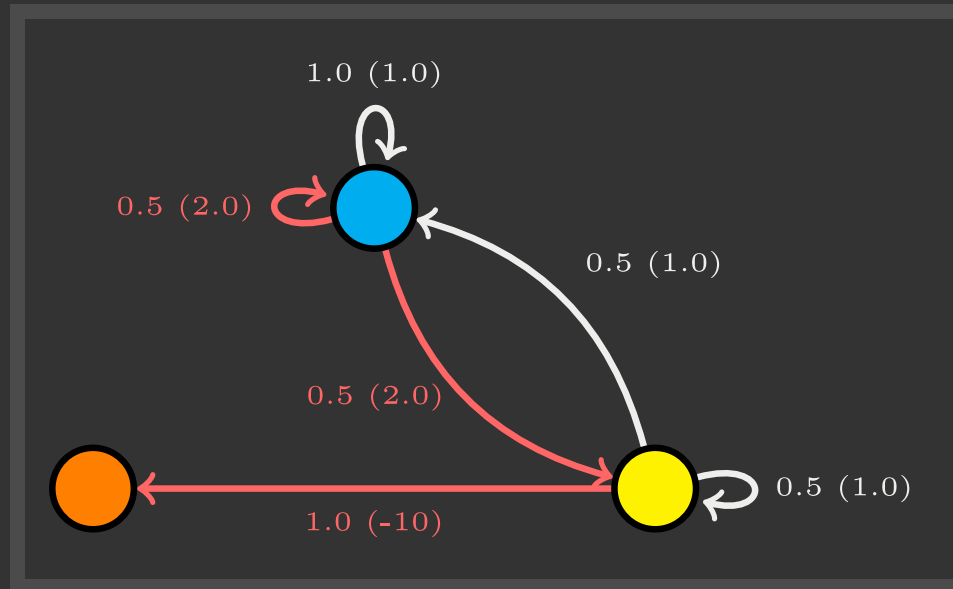


EXAMPLE: CAR RACING

A robot car wants to travel far, quickly

Three states: Cool, Warm, Overheated

Two actions: Slow, Fast



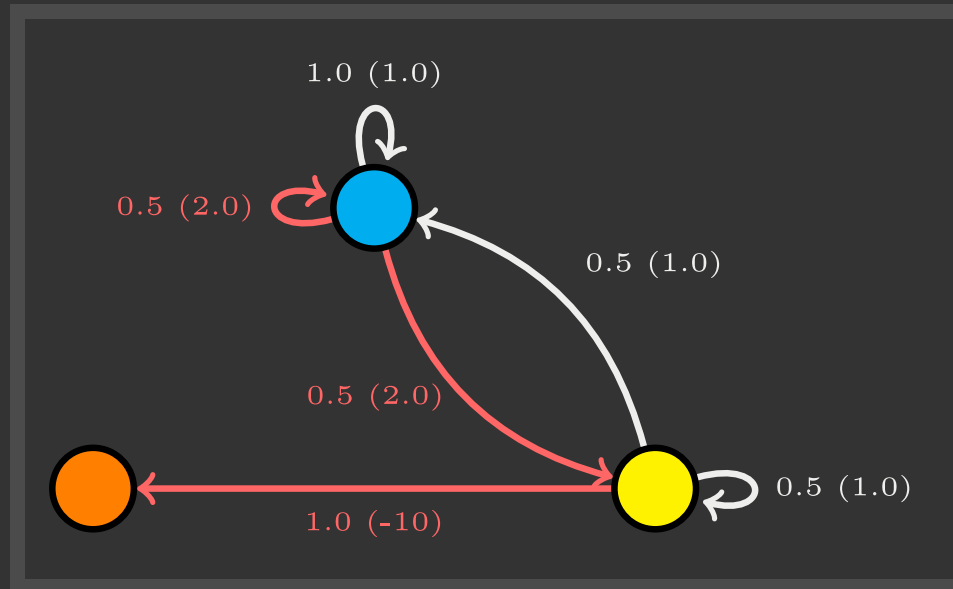
EXAMPLE: CAR RACING

A robot car wants to travel far, quickly

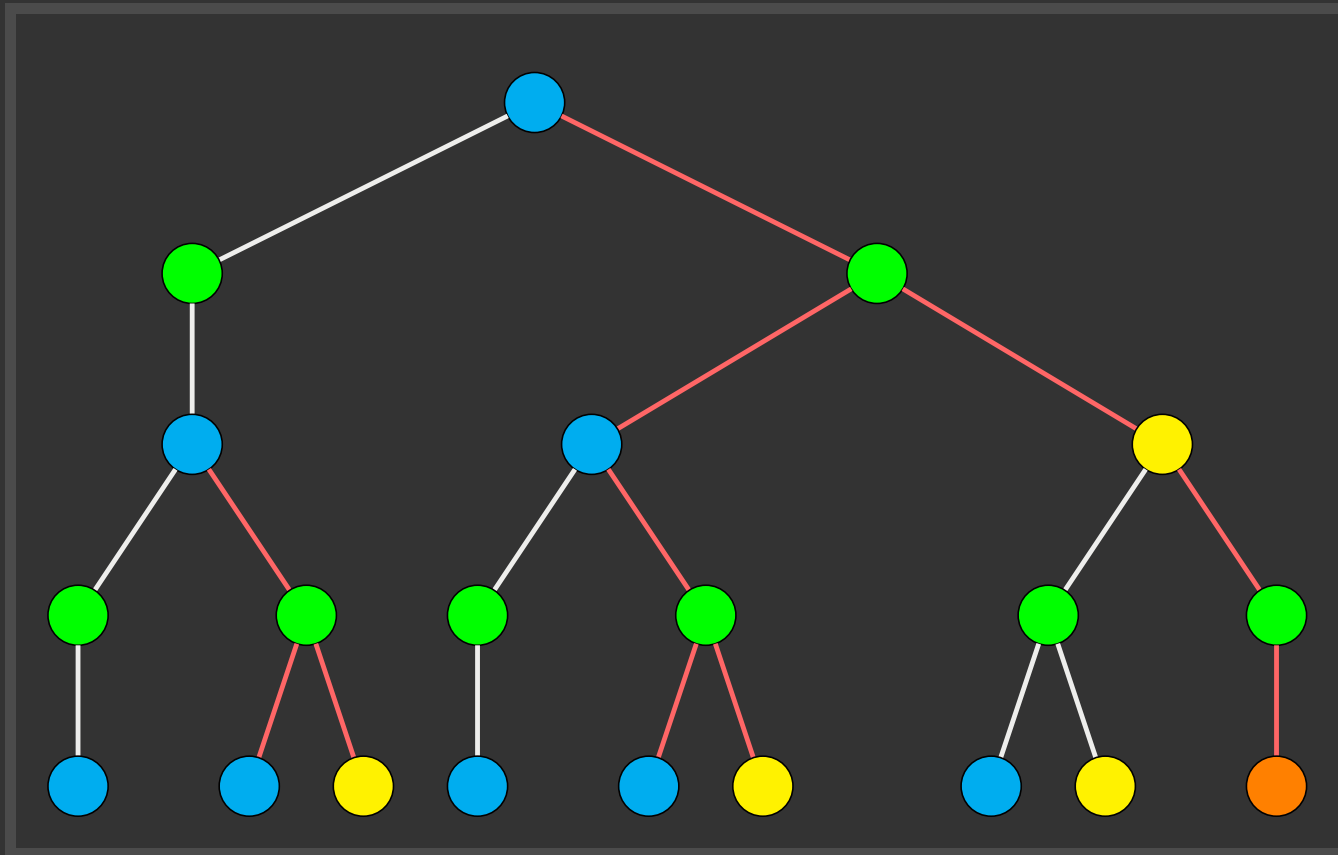
Three states: Cool, Warm, Overheated

Two actions: Slow, Fast

Going faster gets double reward

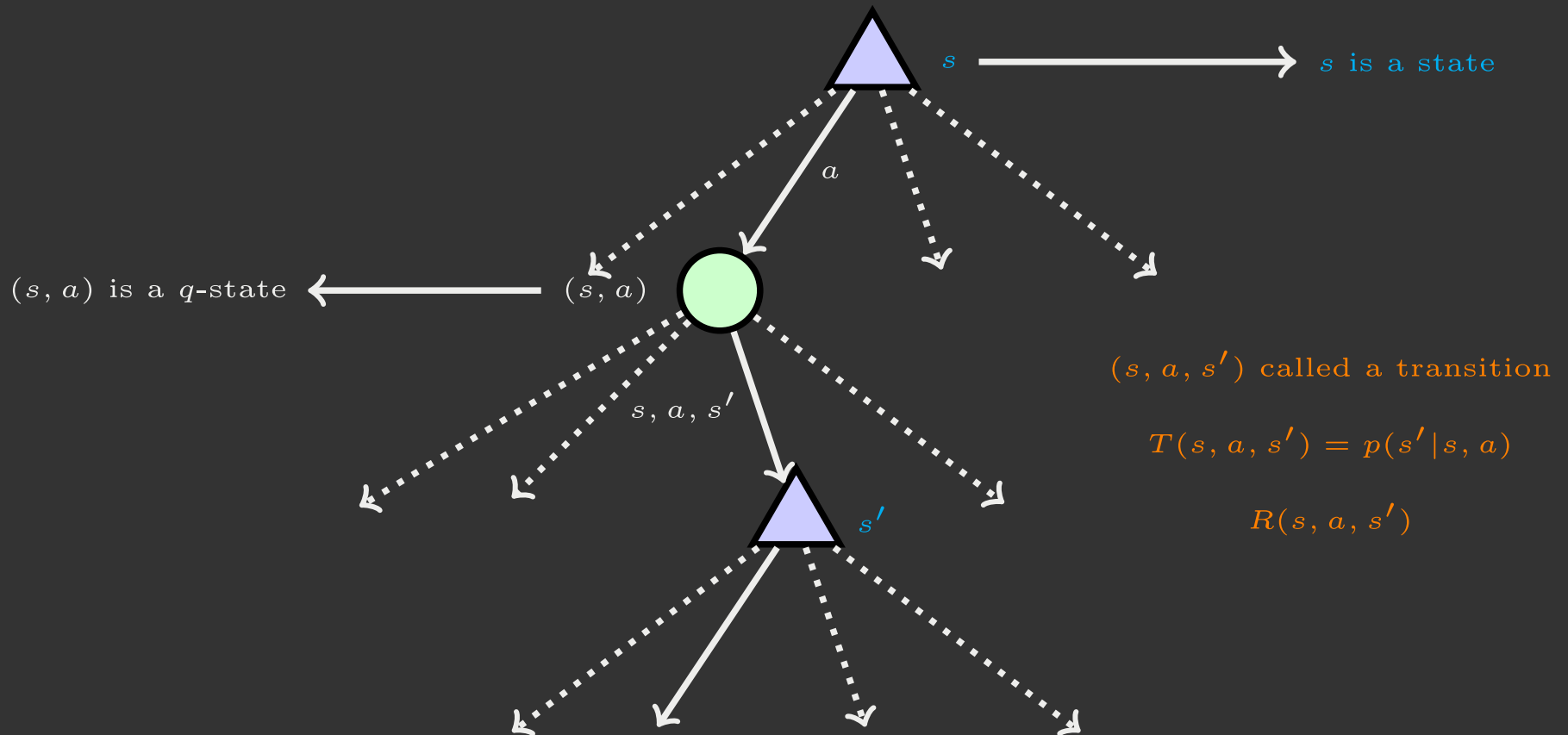


RACING SEARCH TREE



MDP SEARCH TREE

Each MDP state projects an expectimax-like search tree



UTILITY OF SEQUENCES



UTILITY OF SEQUENCES

What preferences should an agent have over reward sequences?



UTILITY OF SEQUENCES

What preferences should an agent have over reward sequences?

More or less? $[1,2,2]$ or $[2,3,4]$



UTILITY OF SEQUENCES

What preferences should an agent have over reward sequences?

More or less? $[1,2,2]$ or $[2,3,4]$

Now or later? $[0,0,1]$ or $[1,0,0]$



DISCOUNTING

It is reasonable to maximize the sum of rewards



DISCOUNTING

It is reasonable to maximize the sum of rewards

It is also reasonable to prefer rewards now to rewards later



DISCOUNTING

It is reasonable to maximize the sum of rewards

It is also reasonable to prefer rewards now to rewards later

One solution:



DISCOUNTING

It is reasonable to maximize the sum of rewards

It is also reasonable to prefer rewards now to rewards later

One solution: values of rewards decay exponentially



DISCOUNTING

It is reasonable to maximize the sum of rewards

It is also reasonable to prefer rewards now to rewards later

One solution: values of rewards decay exponentially



Worth Now

DISCOUNTING

It is reasonable to maximize the sum of rewards

It is also reasonable to prefer rewards now to rewards later

One solution: values of rewards decay exponentially



Worth Now



Worth Next Step

DISCOUNTING

It is reasonable to maximize the sum of rewards

It is also reasonable to prefer rewards now to rewards later

One solution: values of rewards decay exponentially



Worth Now



Worth Next Step



Worth in Two Steps

DISCOUNTING

How to discount?



DISCOUNTING

How to discount?

- Each time we descend a level, we multiply in the discount once



DISCOUNTING

How to discount?

- Each time we descend a level, we multiply in the discount once

Why discount?

- Sooner rewards probably do have higher utility than later rewards
- Also helps our algorithms converge



DISCOUNTING

How to discount?

- Each time we descend a level, we multiply in the discount once

Why discount?

- Sooner rewards probably do have higher utility than later rewards
- Also helps our algorithms converge

Example: discount of 0.5

- $U([1, 2, 3]) = 1 * 1 + 0.5 * 2 + 0.25 * 3$
- $U([1, 2, 3]) < U([3, 2, 1])$



STATIONARY PREFERENCES

Assumption: if we assume stationary preferences:

$$\begin{array}{ccc} [a_1, a_2, \dots] & \succ & [b_1, b_2, \dots] \\ & \Updownarrow & \\ [r, a_1, a_2, \dots] & \succ & [r, b_1, b_2, \dots] \end{array}$$



STATIONARY PREFERENCES

Assumption: if we assume stationary preferences:

$$\begin{array}{ccc} [a_1, a_2, \dots] & \succ & [b_1, b_2, \dots] \\ & \Downarrow & \\ [r, a_1, a_2, \dots] & \succ & [r, b_1, b_2, \dots] \end{array}$$

Then: there are only two ways to define utilities



STATIONARY PREFERENCES

Assumption: if we assume stationary preferences:

$$\begin{array}{ccc} [a_1, a_2, \dots] & \succ & [b_1, b_2, \dots] \\ & \Updownarrow & \\ [r, a_1, a_2, \dots] & \succ & [r, b_1, b_2, \dots] \end{array}$$

Then: there are only two ways to define utilities

- Additive utility:

$$U([r_0, r_1, r_2, \dots]) = r_0 + r_1 + r_2 + \dots$$



STATIONARY PREFERENCES

Assumption: if we assume stationary preferences:

$$\begin{array}{ccc} [a_1, a_2, \dots] & \succ & [b_1, b_2, \dots] \\ & \Updownarrow & \\ [r, a_1, a_2, \dots] & \succ & [r, b_1, b_2, \dots] \end{array}$$

Then: there are only two ways to define utilities

- Additive utility:

$$U([r_0, r_1, r_2, \dots]) = r_0 + r_1 + r_2 + \dots$$

- Discounted utility:

$$U([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$$



INFINITE UTILITIES

Problem: What if the game lasts forever? Do we get infinite rewards?



INFINITE UTILITIES

Problem: What if the game lasts forever? Do we get infinite rewards?

Solutions:



INFINITE UTILITIES

Problem: What if the game lasts forever? Do we get infinite rewards?

Solutions:

- Finite horizon: (similar to depth-limited search)



INFINITE UTILITIES

Problem: What if the game lasts forever? Do we get infinite rewards?

Solutions:

- Finite horizon: (similar to depth-limited search)
 - Terminate episodes after a fixed T steps (e.g. life)
 - Gives non-stationary policies (π depends on time left)



INFINITE UTILITIES

Problem: What if the game lasts forever? Do we get infinite rewards?

Solutions:

- Finite horizon: (similar to depth-limited search)
 - Terminate episodes after a fixed T steps (e.g. life)
 - Gives non-stationary policies (π depends on time left)
- Discounting: use $0 < \gamma < 1$

$$U([r_0, \dots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq \frac{R_{max}}{1 - \gamma}$$



INFINITE UTILITIES

Problem: What if the game lasts forever? Do we get infinite rewards?

Solutions:

- Finite horizon: (similar to depth-limited search)
 - Terminate episodes after a fixed T steps (e.g. life)
 - Gives non-stationary policies (π depends on time left)
- Discounting: use $0 < \gamma < 1$

$$U([r_0, \dots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq \frac{R_{max}}{1 - \gamma}$$

- Smaller γ means smaller "horizon" — shorter term focus



INFINITE UTILITIES

Problem: What if the game lasts forever? Do we get infinite rewards?

Solutions:

- Finite horizon: (similar to depth-limited search)
 - Terminate episodes after a fixed T steps (e.g. life)
 - Gives non-stationary policies (π depends on time left)
- Discounting: use $0 < \gamma < 1$

$$U([r_0, \dots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq \frac{R_{max}}{1 - \gamma}$$

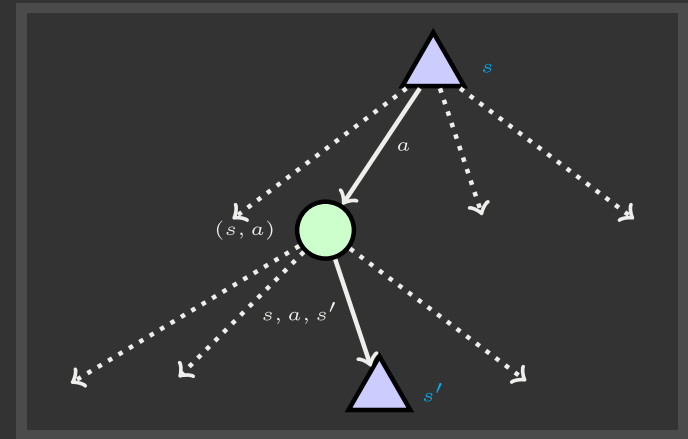
- Smaller γ means smaller "horizon" — shorter term focus
- Absorbing state: guarantee that for every policy, a terminal state will eventually be reached (like "overheated" for racing)



RECAP: DEFINING MDPs

Markov decision processes:

- Set of states $\mathcal{S}$
- Start state s_0
- Set of actions $\mathcal{A}$
- Transitions $P(s'|s, a)$ (or $T(s, a, s')$)
- Rewards $R(s, a, s')$ (and discount γ)



MDP quantities so far:

- Policy = Choice of action for each state
- Utility = sum of (discounted) rewards



3



